

Embeddings: cuando el texto por fin entiende significado

De palabras-símbolo a palabras-vector, y la búsqueda semántica que arregla la falla de dos sesiones

Ramsés Alejandro Camas Nájera

Universidad Politécnica de Chiapas • 9º cuatrimestre • Semana 7, Sesión 3 • Junio 2026

Dos sesiones chocando contra el mismo muro



TF-IDF (Sesión 1)

Palabras como símbolos atómicos. “agua” y “hídrico” no comparten nada.



BM25 (Sesión 2)

Mejor ranking, pero sigue contando cadenas. El significado nunca entró.



K-Means (Sesión 2)

Agrupó por co-ocurrencia de palabras: d02 y d13 quedaron en grupos distintos.

La raíz común: los tres representan cada palabra como una dimensión independiente. Para la máquina, “agua” e “hídrico” son tan distintas como “agua” y “jirafa”. Hoy cambiamos la representación.

Agenda — de la intuición al buscador que entiende



Bloque 1 — La hipótesis distribucional

Por qué el contexto define el significado, y cómo eso permite vectores densos en vez de conteos dispersos.

~45 min



Bloque 2 — word2vec, GloVe, FastText

Cómo se aprenden los embeddings. Por qué FastText es la elección para el español.

~60 min



Bloque 3 — Búsqueda semántica

De palabra a documento, y re-correr los Labs 2 y 4 con embeddings. Bases vectoriales (Qdrant).

~60 min



Cierre — El nuevo límite

Un vector por palabra sin importar el contexto: “banco” financiero vs. “banco” de peces.

~15 min

01

La hipótesis distribucional

La idea de 1950 que sostiene toda la IA del lenguaje moderna: el significado vive en el contexto.

- “You shall know a word by the company it keeps”
- De conteos dispersos a vectores densos
- Qué es un word embedding

El significado está en la compañía que mantiene

“You shall know a word by the company it keeps.” — J.R. Firth, 1957

Palabras que aparecen en contextos parecidos tienden a significar cosas parecidas. No hace falta un diccionario: basta con observar millones de oraciones.

“agua” aparece junto a...

...escasez · pozo · sequía · desabasto · potable · falta de...

“hídrico” aparece junto a...

...escasez · recurso · sequía · desabasto · crisis · falta de...

Comparten casi toda su compañía → deben significar casi lo mismo. Un algoritmo puede aprender eso solo.

De conteos dispersos a vectores densos

Lo que tenían: one-hot / TF-IDF

agua = [0, 0, 1, 0, 0, ..., 0]
hídrico = [0, ..., 1, 0, 0, 0, 0]

- Miles de dimensiones ($|V|$), casi todas cero.
- Cada palabra es ortogonal a las demás: similitud = 0 siempre.
- La maldición de la dimensionalidad que sufrieron en K-Means.

Lo nuevo: embeddings densos

agua = [0.21, -0.4, ..., 0.7]
hídrico = [0.19, -0.38, ..., 0.66]

- Pocas dimensiones (50–300), todas con valor.
- Aprendidas del contexto: palabras similares → vectores cercanos.
- agua e hídrico quedan casi encimadas → coseno alto.

“Embedding” = incrustar el vocabulario en un espacio continuo donde la cercanía geométrica significa cercanía de significado.

El espacio captura relaciones, no solo cercanía

Si el espacio codifica significado, las direcciones también significan. La aritmética vectorial revela analogías:

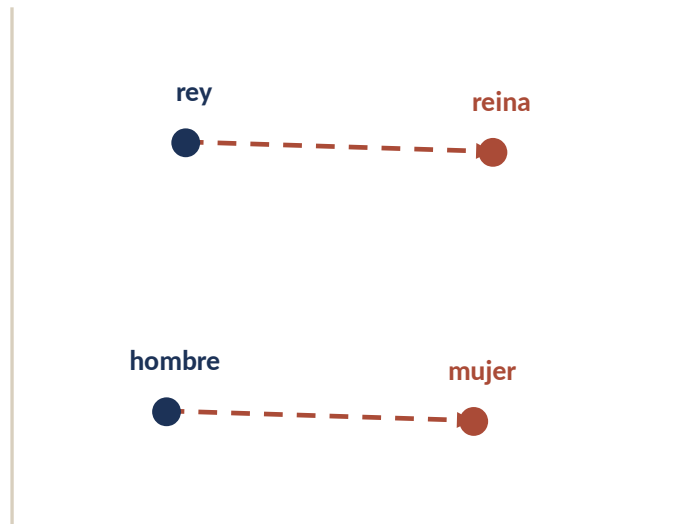
rey - hombre + mujer $\approx$ reina

la dirección “género” es una constante en el espacio

Otras relaciones que emergen solas:

París - Francia + México $\approx$ CDMX

mejor - bueno + malo $\approx$ peor



vectores paralelos = misma relación

02

word2vec, GloVe, FastText

Tres formas de aprender estos vectores a partir de texto crudo, sin una sola etiqueta humana.

- word2vec: predecir el contexto
- GloVe: co-ocurrencia global
- FastText: n-gramas de caracteres
- Cuál elegir para el español

word2vec: aprende prediciendo el contexto

Una red simple recorre el texto con una ventana y aprende a predecir las palabras vecinas. Los pesos que aprende SON los embeddings.



Skip-gram

Dada la palabra objetivo, predice su contexto.

sequía → {prolongada, afectó}

Mejor con palabras raras y corpus chico.

CBOW

Dado el contexto, predice la palabra objetivo.

{prolongada, afectó} → **sequía**

Más rápido; mejor con corpus grande.

El truco: supervisión sin etiquetas

¿De dónde salen las etiquetas para entrenar? Del propio texto. Cada ventana genera ejemplos automáticamente: la palabra y sus vecinos. Nadie etiqueta nada — el corpus se supervisa a sí mismo.



Datos infinitos y gratis

Cualquier texto sirve: Wikipedia, noticias, libros. Miles de millones de ejemplos sin costo de anotación.



Aprende sin saber qué significa

La red nunca ve definiciones. Solo co-ocurrencias. Aun así, el significado emerge en la geometría.



El mismo principio que los LLMs

Predecir la siguiente palabra (o las vecinas) es la receta que escaló hasta GPT. Esto es su ancestro directo.

GloVe: aprender de la co-ocurrencia global

word2vec aprende ventana por ventana (local). GloVe construye primero la matriz global de co-ocurrencias de todo el corpus y la factoriza.

La idea

- Cuenta cuántas veces cada par de palabras aparece junto en todo el corpus.
- Aprende vectores tales que su producto punto aproxima el log de esa co-ocurrencia.
- Combina lo mejor de los métodos de conteo (TF-IDF) y los predictivos (word2vec).

Conexión con lo que ya saben

- La matriz de co-ocurrencia es prima de la matriz documento $\times$ término del Lab 2.
- Factorizar una matriz para obtener vectores densos es reducción de dimensionalidad — lo mismo que hace PCA.
- GloVe = “Global Vectors”, por mirar todo el corpus a la vez.

FastText: la elección para el español

La diferencia clave: FastText no aprende un vector por palabra, sino por n-gramas de caracteres. “sequía” = <se, seq, equ, quí, uía, ía> + la palabra completa. El vector final es la suma de sus partes.



Resuelve la morfología del español

“corro”, “corriendo”, “corrí” comparten n-gramas → vectores cercanos automáticamente. Justo el problema que motivó stemming vs. lematización en la Sesión 1.



Maneja palabras nunca vistas (OOV)

“tuxtleco” no estaba en el entrenamiento, pero sus n-gramas sí: FastText le construye un vector. word2vec y GloVe simplemente fallan.

Para lenguas morfológicamente ricas como el español, FastText suele ser el punto de partida correcto. Hay vectores preentrenados en español (FastText cc.es, SBWC).

Cuál usar: comparativa rápida

	word2vec	GloVe	FastText
Aprende de	ventanas locales	co-ocurrencia global	n-gramas de caracteres
Palabras OOV	falla	falla	las construye
Morfología rica	débil	débil	fuerte
Velocidad	rápido	medio	rápido
Mejor para...	<i>propósito general</i>	<i>corpus muy grande</i>	<i>español y lenguas flexivas</i>

Regla práctica: para proyectos en español, empiecen con FastText preentrenado y solo entrenen el suyo si tienen un corpus de dominio grande.

Por qué esto arregla la falla del agua

“agua” e “hídrico” comparten casi todo su contexto (escasez, sequía, desabasto), así que el modelo les aprende vectores casi idénticos.

ANTES (TF-IDF)



$$\text{coseno}(\text{agua}, \text{hídrico}) = 0$$

Cadenas distintas → vectores ortogonales → cero relación. La consulta nunca cruzaba el puente.

AHORA (embeddings)



$$\text{coseno}(\text{agua}, \text{hídrico}) \approx 0.8$$

Contextos compartidos → vectores cercanos → el modelo sabe que hablan de lo mismo.

La representación cambió, y con ella todo lo que construimos encima: búsqueda, ranking y clustering por fin ven significado.

Lab 5a — Explorar embeddings en español



Carguen vectores FastText preentrenados en español y exploren el espacio. Sin entrenar nada todavía.

1

Cargar vectores — FastText es (gensim o la librería oficial); inspeccionar dimensión y tamaño del vocabulario.

2

Vecinos más cercanos — `most_similar('sequía'), ('café'), ('chiapas')`: ¿tienen sentido los vecinos? Documenten sorpresas.

3

Probar la falla — `similarity('agua', 'hídrico')` y `('agua', 'jirafa')`: comprueben que el primero es alto y el segundo bajo.

4

Analogías — `rey - hombre + mujer`; `méxico - cdmx + chiapas`. ¿Qué devuelve? ¿Cuándo falla la analogía?

Defensa: encuentren un par de palabras donde el embedding se equivoque (vecinos sin sentido) y propongan por qué.

BLOQUE

03

Búsqueda semántica

De vectores de palabra a vectores de documento, y el buscador que por fin entiende la intención.

- De palabra a documento
- El pipeline de semantic search
- Re-correr los Labs 2 y 4
- Bases vectoriales y ANN (Qdrant)

De palabra a documento

Tenemos un vector por palabra. Para buscar y agrupar necesitamos un vector por documento. La forma más simple: promediar.

Promedio de vectores

```
doc = mean(vec(t) for t in doc)
```

- Rápido y sorprendentemente competitivo como baseline.
- Pierde el orden de las palabras (igual que Bag of Words).
- “no hay agua” y “hay agua” quedan casi idénticos: el promedio no capta negación ni sintaxis.

La mejor forma (Sesión 4)

- Modelos que producen directamente un vector de oración o documento entrenado para similitud (Sentence-BERT).
- Captan contexto y orden, no solo el promedio de palabras.
- Los veremos la próxima sesión, al hacer fine-tuning de BERT.

El pipeline de búsqueda semántica



1 • Indexar

Cada documento del corpus → vector.
Se guarda una vez.



2 • Consulta

El texto de la consulta → vector, con el
mismo modelo.



3 • Vecinos

Documentos más cercanos por coseno
= los más relevantes.

La diferencia con el Lab 2: el mecanismo es idéntico — vectores y coseno — pero los vectores ahora codifican significado en vez de conteos de palabras. Misma maquinaria, representación más rica. **Lo único nuevo es de dónde salen los vectores.**

“problemas de agua” — ahora con embeddings

Lab 2 • TF-IDF

```
d13  agua potable      ✓  
d14  concurso robótica x  
d12  feria café/cacao  x  
–    crisis hídrica    NO APARECE
```

coseno(agua, hídrico) = 0

Lab 5 • Embeddings

```
d13  agua potable      ✓  
d02  crisis hídrica    ✓  
d04  sequía y cultivos ✓  
d01  inundaciones      ✓
```

encuentra el significado, no la palabra

La consulta nunca dijo “hídrica”. El buscador lo entendió de todos modos. Eso es búsqueda semántica.

Y el clustering por fin agrupa por significado

Re-corran el Lab 4 con embeddings de documento en vez de TF-IDF. Los grupos cambian — para bien.

Lab 4 • TF-IDF

d02 (crisis hídrica) → grupo “sequía”

d13 (agua potable) → grupo “servicios”

Dos documentos de agua, separados por usar palabras distintas.

Lab 4 • Embeddings

d02 (crisis hídrica) → grupo “agua”

d13 (agua potable) → grupo “agua”

Por fin juntos: el modelo entiende que ambos tratan del mismo tema.

La misma falla que perseguimos en búsqueda Y en clustering se cierra con un solo cambio: la representación.

Cuando el corpus crece: bases vectoriales y ANN

Comparar la consulta contra cada documento (fuerza bruta) funciona con 14 documentos. Con millones, es inviable: hay que indexar.



El problema de escala

Fuerza bruta = $O(N \cdot d)$ por consulta. Con millones de vectores de 300 dimensiones, cada búsqueda se vuelve lentísima.



ANN: vecinos aproximados

Índices como HNSW encuentran vecinos casi-óptimos en tiempo sublineal, sacrificando un poco de exactitud por mucha velocidad.



Bases de datos vectoriales

Qdrant, FAISS, Milvus: almacenan embeddings, indexan con ANN y sirven búsquedas por similitud a escala de producción.

Qdrant es la base vectorial del stack de esta unidad. En la Unidad 3 será el motor de recuperación de un sistema RAG.

Lab 5b — Buscador semántico sobre su corpus



Reemplacen los vectores TF-IDF del Lab 2 por embeddings de documento y comparen los tres sistemas.

1

Embeber el corpus — vector de cada documento = promedio de los vectores FastText de sus términos (reusen su preprocesamiento).

2

Buscar por coseno — `buscar_semantico(consulta, k=5)`: embeben la consulta y devuelven los vecinos más cercanos.

3

Comparar 3 sistemas — para sus 5 consultas del Lab 3: TF-IDF vs. BM25 vs. semántico, lado a lado. ¿Qué encuentra cada uno?

4

Re-evaluar con sus métricas — corran las métricas del Lab 3 sobre el buscador semántico. ¿Mejora el nDCG en las consultas de significado?

Sus métricas del Lab 3 ahora miden el salto semántico: esperen mejoras justo en las consultas que antes fallaban.

Un vector por palabra... ¿siempre el mismo?

“Me senté en el banco del parque”

banco = asiento

“Fui al banco a sacar dinero”

banco = institución financiera

Pero FastText le da a “banco” UN solo vector — el mismo en las dos frases.

Los embeddings estáticos ignoran el contexto de la oración: la polisemia los derrota. Necesitamos un vector que cambie según la frase — embeddings contextuales. Eso es BERT, y es la próxima sesión.

Lo que sigue

Próxima sesión

- BERT y embeddings contextuales: un vector distinto por palabra según la oración.
- Fine-tuning: adaptar un modelo gigante preentrenado a tareas con poco dato.
- Tres usos: embeddings de oración, clasificación y NER.

Para antes de la próxima clase

- Terminar Lab 5a y 5b y subirlos al repo (con AI_USAGE.md).
- Lectura: Jurafsky & Martin (SLP3) cap. 6 (vector semantics) y Mikolov et al. 2013.
- Traer un caso de polisemia donde su buscador semántico se confunda.

La Unidad 2 cierra con un buscador que entiende significado. La Unidad 3 lo lleva a producción: transformers, RAG y MLOps.

Bibliografía de la sesión

-  **Mikolov, T. et al. (2013).** Efficient Estimation of Word Representations in Vector Space (word2vec). arXiv:1301.3781.
-  **Pennington, J., Socher, R. & Manning, C. (2014).** GloVe: Global Vectors for Word Representation. EMNLP.
-  **Bojanowski, P. et al. (2017).** Enriching Word Vectors with Subword Information (FastText). TACL, 5.
-  **Jurafsky, D. & Martin, J. H. (2026).** Speech and Language Processing (3ª ed., borrador). — Cap. 6: Vector Semantics and Embeddings.
-  **Cardellino, C. (2019).** Spanish Billion Words Corpus and Embeddings (SBWC). — Vectores preentrenados en español.